

NATIONAL UNIVERSITY OF SINGAPORE
SCHOOL OF COMPUTING

PLACEMENT EXAMINATION
FOR CS1010 (AND EQUIVALENT MODULES)

8 July 2020

Time Allowed: 2 hours

INSTRUCTIONS TO CANDIDATES

1. This assessment paper contains **9** questions and comprises **9** printed pages.
2. Answer **ALL** questions in the **ANSWER SHEETS**.
3. This is an **OPEN BOOK** assessment. The total mark is 100.
4. Please write your **Student Number** clearly on the first page of your **ANSWER SHEETS**. Do not write your name.
5. Please shade one of the circles on the first page of your **ANSWER SHEETS** to indicate which module you are applying this Placement Exam for.
6. The distribution of marks for the questions in this paper is as follows:

Question	Marks
1	5
2	5
3	10
4	20
5	5
6	10
7	8
8	12
9	25
Total	100

Section A

- KenKen** is a puzzle created by a Japanese math teacher Tetsuya Miyamoto in 2004. It resembles Sudoku in some way. A KenKen puzzle consists of a grid consisting of $n \times n$ cubes ($3 \leq n \leq 9$). Each puzzle is further divided into cages, which are groups of cubes. A cage can be as small as one cube. Each cage contains a target number and an operation (+, -, $\times$, $\div$). To solve the puzzle, you must place the correct number in each cube, guided by the target numbers and their specified math operation. Moreover, each horizontal row and each vertical column must be filled with numbers 1 to n without repeating any of them.

An example of a 4×4 puzzle (Figure 1(a)) and its solution (Figure 1(b)) are shown below. There are seven cages, each marked by bold boundary. In the solution, the $8+$ cage consists of 4, 3 and 1, which add up to 8. Note that every row (and column) must contain the numbers 1 to n without repetition.

$2 \div$		$3 \times$	
$8 +$			$6 \times$
	$2 -$		
$2 -$		$2 \div$	

Figure 1(a)

$2 \div$	2	4	$3 \times$	3	1
$8 +$	4	3	1	$6 \times$	2
	1	$2 -$	2	4	3
$2 -$	3	1	$2 \div$	2	4

Figure 1(b)

Solve the KenKen 5×5 Puzzle in Figure 1(c) below.

[5 marks]

$4 -$		$60 \times$	$2 \div$	
$2 -$				$1 -$
$13 +$		5	$20 \times$	
	$1 -$	$6 \times$		
				1

Figure 1(c)

You are to write your answer as a 5×5 matrix, with values separated by commas. For example, the solution in Figure 1(b) can be written as follows:

2, 4, 3, 1
 4, 3, 1, 2
 1, 2, 4, 3
 3, 1, 2, 4

2. [5 marks]

On an island, the islanders are of two types: **knights** and **knaves**. Knights always tell the truth, while knaves always lie. Identify the islanders in each of the parts below.



(a) You met two islanders *A* and *B*.

A says, "We are both knaves."

B says nothing.

What are *A* and *B*?

(b) You met two islanders *C* and *D*. What are *C* and *D*?

C says, "At least one of us is a knave."

D says nothing.

(c) You met three islanders *E*, *F* and *G*. What are *E*, *F* and *G*?

E says, "All of us are knaves."

F says, "Exactly one of us is a knight."

G says nothing.

Now, there is a third type of islanders: **spies**. Spies either tell the truth or lie.

(d) You met three islanders *P*, *Q* and *R*. You know that one of them is a knight, one is a knave, and one is a spy. What are *P*, *Q* and *R*?

P says, "I am not a spy."

Q says, "I am a knave."

R says, "If you ask me, I would say *Q* is a spy."

(e) You met three islanders *X*, *Y* and *Z*. You know that one of them is a knight, one is a knave, and one is a spy. What are *X*, *Y* and *Z*?

X says, "I am the spy."

Y says, "I am the knave."

Z says, "I am the knight."

3. [10 marks]

Logo is an educational programming language designed in 1967 by Wally Feurzeig, Seymour Papert, and Cynthia Solomon. One distinctive feature of Logo is its “Turtle” graphics. The turtle is represented by a triangle on the drawing window. The initial position of the turtle is at the origin (0,0), the centre of the window, with the turtle pointing up (north), as shown below. The coordinate system is the usual 2-dimensional system where (x,y) represents the x-coordinate and y-coordinate respectively.



The turtle can be controlled by the following commands:

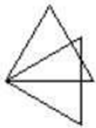
Command	Description
fd x	Move turtle forward x steps
bk x	Move turtle backward x steps
rt x	Turn turtle x degrees right
lt x	Turn turtle x degrees left
pd	Pen down (leave ink on screen as the turtle moves) – default
pu	Pen up (not to leave ink on screen as the turtle moves)
st	Show turtle – default
ht	Hide turtle

The following shows some examples:

 fd 50	 fd 50 rt 90	 fd 50 rt 90 fd 30	 fd 50 rt 90 fd 30 lt 90 fd 20 lt 90 bk 20
 rt 90 fd 80 pu fd 20 pd fd 60		 rt 90 fd 80 pu lt 90 fd 20 pd rt 90 fd 20 ht	

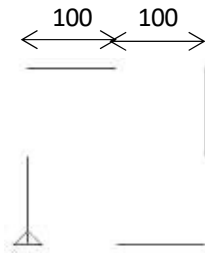
Logo also provides the repeat statement: **repeat n [...]**

to repeat the commands in the brackets n times. The repeat command may be nested. Two examples are given below.

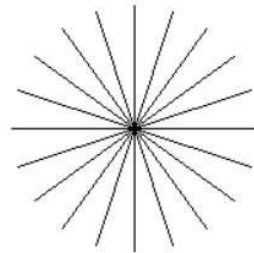
 <pre>rt 30 repeat 3 [fd 50 rt 120] ht</pre>	 <pre>repeat 2 [rt 30 repeat 3 [fd 50 rt 120]] ht</pre>
---	---

Write Logo code to generate each of the following graphics. Your code should be as short as possible, or marks may be deducted. Note that for diagrams (b)-(d), the centre is at the origin and the turtle is hidden. (Diagrams are not drawn to scale.)

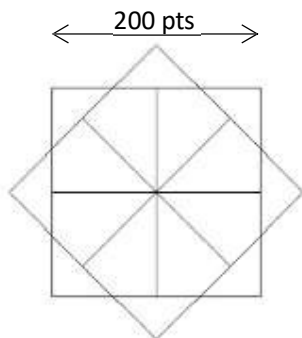
- (a) Four lines each of length 100 points.
Turtle at origin. [2 marks]



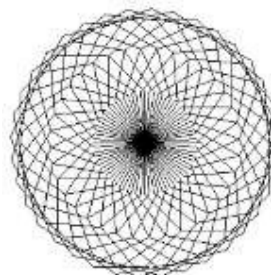
- (b) Twenty lines each of length 120 points
radiating from the origin. [2 marks]



- (c) Two large squares with side 200 points. [3 marks]



- (d) 36 regular octagons arranged in a
circular fashion where the side of each
octagon is 50 points long. [3 marks]

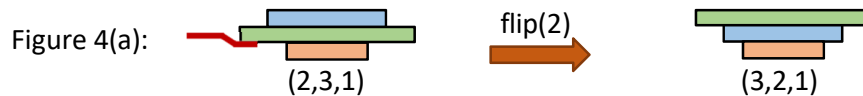


4. [20 marks]

Bill Gates once wrote a paper on the **Pancake-flipping problem**, which goes like this:

The chef in our place is sloppy, and when he prepares a stack of n pancakes, they all come out in different sizes 1 to n , which is unpresentable. Now the waiter may flip the pancakes by grabbing several from the top and flipping them over, repeating this as many times as necessary, so that in the end the stack of pancakes has the smallest pancake at the top, the next smallest below it, and so on, down to the largest at the bottom, before he presents it to the customer. How many flips are needed?

Figure 4(a) shows a stack of 3 pancakes and an example of a flip. The operation **flip(k)** denotes flipping the top k pancakes, where $1 \leq k \leq n$.



Instead of drawing the stack of pancakes, we may represent the configuration of a stack by writing the sizes of the pancakes in the stack from top to bottom. For example, the configuration of the left stack in Figure 4(a) above is (2, 3, 1) and the configuration for the right stack is (3, 2, 1). The aim is to get to the final configuration of (1, 2, 3) for the above example, or (1, 2, 3, ..., n) in general for a stack of n pancakes. To repeat the above flip operations:



(a) How many different configurations are there for 4 pancakes? You do not need to show your working. [2 marks]

(b) Given the following initial configuration of a stack of 5 pancakes and a list of flip operations, write out the configuration after every flip operation. [3 marks]

Initial configuration: (2, 5, 1, 4, 3)

List of flip operations: flip(2), flip(5), flip(2), flip(4), flip(3), flip(3)

(c) Give an initial configuration of a stack of 4 pancakes that requires at least 4 flips to get to the final configuration of (1, 2, 3, 4). List out the flip operations. [3 marks]

(d) Based on the idea in part (b), write an algorithm **pancake(config, n)** to solve the pancake-flipping problem for a stack of n pancakes, that is, to generate a list of flip operations that takes the initial configuration, **config**, to the final configuration (1, 2, 3, ..., n). Your algorithm does not need to generate the fewest number of flip operations. You may assume that **config** is a list of n integers containing 1 to n in any order. A flip operation is called by **flip(config, k)** to flip the top k pancakes. You do not need to write the algorithm for **flip(config, k)**. [10 marks]

(e) Trace the algorithm you have written in part (d) to illustrate how it works on an example of a stack of 4 pancakes. Write down the initial configuration, and list out the flip operations. For each flip operation, write out the next configuration. [2 marks]

Section B

INSTRUCTIONS

- For SM2 students, you are to answer the questions in this section only in C programs.
- For CS students taking CS1101S, you may choose to answer the questions in this section in C/C++, Python or Java.
- Your code will be graded not only for correctness, but also for cohesion and elegance.
- Note that you do not need to perform data input validation.
- When asked to write only a function, you do not need to write the whole program. You may write additional supporting functions if necessary.

5. [5 marks]

An **automorphic number** is a number whose square ends in the same digits as the number itself. For example, 5, 6 and 76 are automorphic numbers as $5^2 = 2\mathbf{5}$, $6^2 = 3\mathbf{6}$, and $76^2 = 57\mathbf{76}$.

Write a function **is_automorphic(n)**, where n is a positive integer, to return true if n is an automorphic number, or false otherwise.

You are not allowed to use any math library function or strings or string functions.

6. [10 marks]

A **complex number** is of the form $a + bi$ where a is the real part and b the imaginary part, a and b are real numbers and $i = \sqrt{-1}$.

Write a complete program to read in two complex numbers x and y and perform division, i.e. x/y . You may assume that the user enters only integer values. You are to check for division by zero error and print appropriate message.

You are to define some structure data type for the complex numbers. For Python programmers, you cannot use any complex package or library, nor built-in complex number type.

A sample run is shown below. Note that a complex number is entered with its real part and imaginary part.

```
Enter 1st complex number: 5 2
Enter 2nd complex number: 1 9
Answer: 0.280488 + -0.524390i
```

The focus of this question is on the use of structure data type and the division operation, not the output format. Hence you do not need to pretty print your complex number in the output. That is, for example, if a complex number is $1.2 - 3.4i$, you may print it as $1.2 + -3.4i$; if it is 1.2 , you may print it as $1.2 + 0i$.

7. [8 marks]

Write a complete program to read a string and convert the string by removing duplicate characters. You may assume that the string has at most 50 characters. Your program should contain a function called **modify**.

You may use string functions and other library functions in your code.

A sample run is shown below. Note that case is sensitive, that is, for example, 'A' is different from 'a'.

```
Enter string      : Zz3gaaG,Aa!zZzA2-333a,01KK.
Modified string: Zz3gaG,A!2-01K.
```

8. [12 marks]

Your math teacher gives you a new workbook and asks you to finish the assigned pages. Although the workbook has a lot of pages, your teacher only asks you to finish certain pages. For example:

100 241 242 243 374 375 480

Isn't it tedious to list out the pages one by one? It is hence better to condense the list into the following format to save space for two or more consecutive pages (using the hyphen '-' to denote a closed range):

100 241-243 374-375 480

Write a complete program to read in the list of pages (see sample run below) and display the list of pages in the condensed format. You may assume the number of pages is at least 1 and at most 1000, and the pages in the workbook start at page 1. The page numbers are given in ascending order and there are no duplicates. You may use any built-in functions.

A sample run is shown below:

```
Enter the number of pages: 7
Enter the pages: 100 241 242 243 374 375 480
Condensed list: 100 241-243 374-375 480
```


9. [25 marks]

(a) An $n \times n$ square matrix is a 2-dimensional array with n rows and n columns.

Write a function **randomize_matrix(mat, n)** to fill an $n \times n$ matrix **mat** with random values that are integers in the range $[1, 100]$, such that no two values in the same (horizontal) row are identical. You may assume that $1 \leq n \leq 10$. [10 marks]

Figure 9(a) shows an invalid 4×4 matrix (there are two 19s in the last row) and Figure 9(b) shows a valid 4×4 matrix.

12	42	79	15
60	55	98	65
7	42	32	13
19	64	19	87

Figure 9(a)

12	42	79	15
60	55	98	65
7	42	32	13
98	64	19	87

Figure 9(b)

(b) Write a function **max_total(mat, n)** to find the largest sum of a path in the given $n \times n$ matrix **mat** from the top-left element (`mat[0][0]`) to the bottom-right element (`mat[n-1][n-1]`) on the condition that you may only move right or down at each step. [15 marks]

For example, given the 4×4 matrix in Figure 9(c) below, the largest sum is 396, obtained from the highlighted path ($12 + 42 + 79 + 98 + 65 + 13 + 87 = 396$). You do not need to produce the path; you need only to produce the largest sum.

12	42	79	15
60	55	98	65
7	42	32	13
98	64	19	87

Figure 9(c)

--- END OF PAPER ---